

Aspect-oriented Spring (AOP)

Practical Implementation: Apply cross-cutting concerns such as logging, security, and transaction management using AOP.

Example: Create aspects to intercept method invocations and apply additional behavior, such as logging method calls.

Practical No.: Aspect-Oriented Spring (AOP)

Title

Implementation of Aspect-Oriented Programming (AOP) in Spring Framework

Aim

To implement **Spring AOP** for handling cross-cutting concerns such as **logging, security, and transaction management** by creating an aspect that intercepts method execution and performs logging before and after the business method.

Objective

After completing this practical, students will be able to:

1. Understand the concept of Aspect-Oriented Programming (AOP).
 2. Learn the importance of cross-cutting concerns.
 3. Implement Spring AOP using annotations.
 4. Create an Aspect class using `@Aspect`.
 5. Apply logging using `@Before` and `@After` advice.
 6. Reduce code duplication by separating logging from business logic.
-

Software Requirements

- JDK 17 or above
 - IntelliJ IDEA / Eclipse
 - Apache Maven
 - Spring Framework
 - Spring AOP
 - AspectJ Weaver
-

Theory

In large enterprise applications, some functionalities are required in many classes, such as:

- Logging
- Security
- Transaction Management
- Exception Handling
- Performance Monitoring

These functionalities are called **Cross-Cutting Concerns** because they affect multiple parts of the application.

Instead of writing the same logging code inside every method, **Spring AOP** allows us to define it once in an **Aspect**, and Spring automatically executes it whenever the target method is called.

This keeps the business logic clean, reusable, and easier to maintain.

Real-Life Scenario

Consider an **Online Banking System**.

When a customer transfers money:

- Verify user authentication.
- Record the transaction.
- Start a database transaction.
- Execute the money transfer.
- Commit the transaction.
- Log the operation.

The **money transfer** is the business logic, while logging, authentication, and transaction management are cross-cutting concerns handled using Spring AOP.

Project Structure

```
SpringAOPExample
├── src
│   └── main
│       ├── java
│       │   ├── com.example.config
│       │   │   └── AppConfig.java
│       │   ├── com.example.service
│       │   │   └── StudentService.java
│       │   ├── com.example.aspect
│       │   │   └── LoggingAspect.java
│       │   └── com.example
│       │       └── MainApp.java
│       └── pom.xml
```

pom.xml Dependencies

```
<dependencies>

    <!-- Spring Context -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>6.1.8</version>
    </dependency>

    <!-- Spring AOP -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-aop</artifactId>
        <version>6.1.8</version>
    </dependency>

    <!-- AspectJ Weaver -->
    <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjweaver</artifactId>
        <version>1.9.22</version>
    </dependency>

</dependencies>
```

Step 1: StudentService.java

```

package com.example.service;

import org.springframework.stereotype.Component;

@Component
public class StudentService {

    public void displayMessage() {

        System.out.println("Welcome to Spring AOP Practical");

    }

    public void calculate() {

        int sum = 10 + 20;

        System.out.println("Addition = " + sum);

    }

}

```

Step 2: LoggingAspect.java

```

package com.example.aspect;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.example.service.StudentService.*(..))")
    public void beforeMethod(JoinPoint jp) {

        System.out.println("-----");
        System.out.println("LOG : Method Started");
        System.out.println("Method Name : " + jp.getSignature().getName());
        System.out.println("-----");

    }

    @After("execution(* com.example.service.StudentService.*(..))")
    public void afterMethod(JoinPoint jp) {

        System.out.println("-----");
        System.out.println("LOG : Method Completed");
        System.out.println("Method Name : " + jp.getSignature().getName());
        System.out.println("-----");

    }

}

```

Step 3: AppConfig.java

```
package com.example.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableAspectJAutoProxy;

@Configuration
@ComponentScan(basePackages = "com.example")
@EnableAspectJAutoProxy
public class AppConfig {

}
```

Step 4: MainApp.java

```
package com.example;

import com.example.config.AppConfig;
import com.example.service.StudentService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class MainApp {

    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        StudentService service =
            context.getBean(StudentService.class);

        service.displayMessage();

        System.out.println();

        service.calculate();

    }

}
```

Program Output

```
-----
LOG : Method Started
Method Name : displayMessage
-----
```

Welcome to Spring AOP Practical

```
-----  
LOG : Method Completed  
Method Name : displayMessage  
-----
```

```
-----  
LOG : Method Started  
Method Name : calculate  
-----
```

Addition = 30

```
-----  
LOG : Method Completed  
Method Name : calculate  
-----
```

Explanation

1. `@Aspect` marks `LoggingAspect` as an Aspect class.
2. `@Component` registers the aspect as a Spring Bean.
3. `@EnableAspectJAutoProxy` enables Spring AOP.
4. `@Before` advice runs **before** the target method.
5. `@After` advice runs **after** the target method.
6. The pointcut expression:

```
execution(* com.example.service.StudentService.*(..))
```

matches all methods in the `StudentService` class.

7. `JoinPoint` provides runtime information such as the method name being executed.
-

Viva Questions

1. What is Aspect-Oriented Programming (AOP)?
2. What are cross-cutting concerns?
3. What is an Aspect?
4. What is Advice?
5. What is a Pointcut?
6. What is a Join Point?
7. What is the purpose of `@EnableAspectJAutoProxy`?
8. What is the difference between `@Before` and `@After` advice?
9. Why is logging implemented using AOP?
10. Name some real-world applications of Spring AOP.

Result

The Spring AOP application was successfully implemented. The aspect intercepted the service methods and executed logging before and after method execution without modifying the business logic, demonstrating the separation of cross-cutting concerns using Aspect-Oriented Programming.